

Кратко в предыдущих сериях

Что советуем перед семинаром освежить в памяти из курса статистики:

- Описательные статистики: среднее, частота абсолютная и относительная, их поиск с помощью пакета `numpy`
- Генеральная совокупность, выборка
- Случайные величины, независимость случайных величин, их распределение
- Метод Монте-Карло
- Матожидание и дисперсия
- Точечные и интервальные оценки
- Оценки параметров среднего и пропорции

Тема 1. Интро.

0. Цель и задачи

Цель — научиться не попадаться на статистические заблуждения и решать некоторые задачи с собеседований.

Задачи:

- вспомнить статистику (пункты выше)
- познакомиться со статистическими парадоксами
- научиться моделировать некоторые из них
- научиться видеть подвох на данных и не делать некорректные выводы

1. Предисловие

1.1 Ценность этого семинара

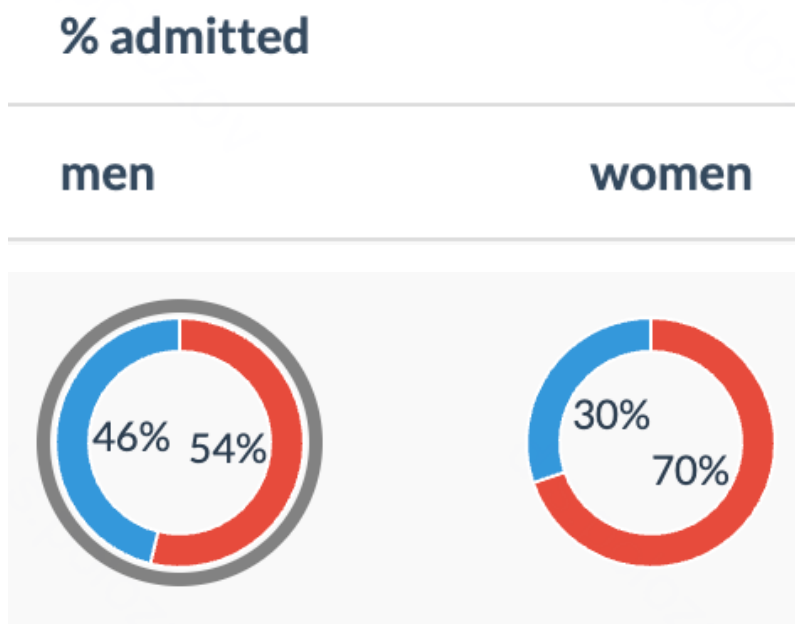
Ценность этого семинара в том, чтобы показать, что аналитика — это не просто подсчет циферок, изображение их на графиках и тривиальные выводы. Это нечто более сложное и творческое. Не всегда выводы, которые кажутся очевидно правдивыми, таковыми являются. Мы рассмотрим несколько примеров и в этом убедимся.

2. Статистические парадоксы

2.1. Парадокс Симпсона

На круговой диаграмме ниже представлены проценты прохождения в Калифорнийский университет Berkeley при поступлении в 1976 году. В связи с этим на Berkeley был подан судебный иск за половую дискриминацию.

Что мы видим? — девушки значительно реже проходят отбор в школу, чем юноши: 30% против 46%.



Вопрос в зал: Есть ли дискриминация и почему? Что же на самом деле происходит?

► Сведения

Продemonстрируем парадокс в действии: упростим ситуацию до 2 факультетов: "легкий" и "сложный". Пусть девушки больше стремятся пройти на сложный факультет, но при этом учатся лучше на легком, чем на сложном. Попробуем так

Ввод [2]:

```
1 import numpy as np
2 import scipy.stats as sps
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 from tqdm.notebook import tqdm
6
7 import seaborn as sns
8 sns.set(font_scale=1, palette='Set2')
9 sns.set_style('whitegrid')
```

```

Ввод [3]: 1 simpson_df = pd.DataFrame(data={
2         'gender': ['male', 'female'],
3         'applied_cnt_easy': [2_000, 500],
4         'admitted_conversion_easy': [0.7, 0.8],
5         'applied_cnt_hard': [500, 2_000],
6         'admitted_conversion_hard': [0.4, 0.5],
7         'applied_cnt_overall': [2_500, 2_500],
8         'admitted_conversion_overall': [1_600 / 2_500, 1_400 / 2_500]
9     })
10 s = simpson_df.style
11
12 s.set_table_styles([ # create internal CSS classes
13     {'selector': '.true', 'props': 'background-color: #e6ffe6;'},
14     {'selector': '.false', 'props': 'background-color: #ffe6e6;'}],
15 ], overwrite=False)
16 cell_color = pd.DataFrame([['false', 'false', 'true'],
17                             ['true', 'true', 'false']],
18                             index=simpson_df.index,
19                             columns=simpson_df.columns[[2, 4, 6]])
20 s.set_td_classes(cell_color)
21 s

```

Out [3]:

	gender	applied_cnt_easy	admitted_conversion_easy	applied_cnt_hard	admitted_conversion_
0	male	2000	0.700000	500	0.40
1	female	500	0.800000	2000	0.50

Визуализируем это в виде pie-диаграмм

Ввод [4]:

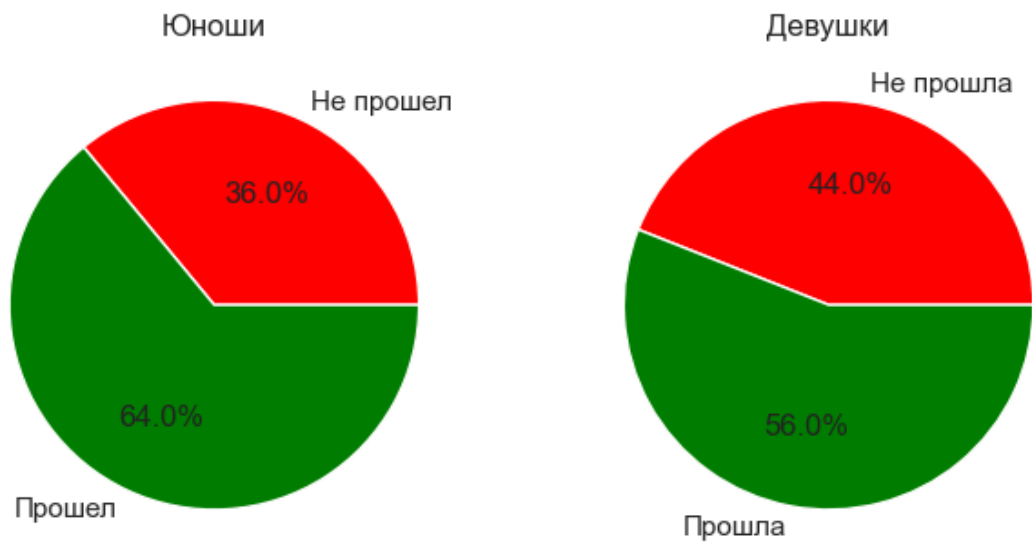
```

1 fig, axes = plt.subplots(1, 4, figsize=(12, 3))
2 axes[0].pie([simpson_df.loc[0, 'applied_cnt_easy'] * (1 - simpson_
3             simpson_df.loc[0, 'applied_cnt_easy'] * simpson_df.loc[0,
4             colors = ['red', 'green'], labels=['Не прошел', 'Прошел'],
5 axes[0].set_title('Юноши, легкий')
6 axes[1].pie([simpson_df.loc[1, 'applied_cnt_easy'] * (1 - simpson_
7             simpson_df.loc[1, 'applied_cnt_easy'] * simpson_df.loc[1,
8             colors = ['red', 'green'], labels=['Не прошла', 'Прошла'],
9 axes[1].set_title('Девушки, легкий')
10
11 axes[2].pie([simpson_df.loc[0, 'applied_cnt_hard'] * (1 - simpson_
12             simpson_df.loc[0, 'applied_cnt_hard'] * simpson_df.loc[0,
13             colors = ['red', 'green'], labels=['Не прошел', 'Прошел'],
14 axes[2].set_title('Юноши, сложный')
15 axes[3].pie([simpson_df.loc[1, 'applied_cnt_hard'] * (1 - simpson_
16             simpson_df.loc[1, 'applied_cnt_hard'] * simpson_df.loc[1,
17             colors = ['red', 'green'], labels=['Не прошла', 'Прошла'],
18 axes[3].set_title('Девушки, сложный')
19 plt.suptitle('Процент прохождения по факультетам')
20 plt.show()
21
22 fig, axes = plt.subplots(1, 2, figsize=(8, 4))
23 axes[0].pie(
24     [simpson_df.loc[0, 'applied_cnt_overall'] * (1 - simpson_df.loc[0,
25     simpson_df.loc[0, 'applied_cnt_overall'] * simpson_df.loc[0,
26     colors = ['red', 'green'], labels=['Не прошел', 'Прошел'], ax
27 axes[0].set_title('Юноши')
28 axes[1].pie(
29     [simpson_df.loc[1, 'applied_cnt_overall'] * (1 - simpson_df.loc[1,
30     simpson_df.loc[1, 'applied_cnt_overall'] * simpson_df.loc[1,
31     colors = ['red', 'green'], labels=['Не прошла', 'Прошла'], ax
32 axes[1].set_title('Девушки')
33 plt.suptitle('Процент прохождения в сумме')
34 plt.show()

```



Процент прохождения в сумме



2.2. Парадокс Монти-Холла

Представьте, что вы стали участником игры, в которой вам нужно выбрать одну из трёх дверей. За одной из дверей находится автомобиль, за двумя другими дверями — козы. Вы выбираете одну из дверей, например, номер 1, после этого ведущий, который знает, где находится автомобиль, а где — козы, открывает одну из оставшихся дверей, например, номер 3, за которой находится коза. После этого он спрашивает вас — не желаете ли вы изменить свой выбор и выбрать дверь номер 2?

Вопрос в зал: Примете ли вы предложение ведущего или не измените свой выбор?

На первый взгляд кажется, что без разницы, менять дверь или нет: двери осталось 2, значит, за каждой из дверей вероятность встретить автомобиль 50%.

Вопрос в зал: Примете ли вы предложение ведущего или не измените свой выбор?

► Сведения

Вы уже знакомы с методом Монте-Карло, так давайте просимулируем игру на python! Пусть автомобиль находится за случайной дверью и игрок тоже выбирает случайную дверь. Сначала протестируем стратегию без смены двери, а потом — со сменой и сравним результаты.

Практика: вставьте недостающие части кода

Ввод []:

```

1 def monty_hall_game(change=False, n_iters=10_000):
2     '''
3     Функция, симулирующая игру Монти-Холла с помощью метода Монте-
4
5     Параметры:
6     - change: bool, default=False — параметр, отвечающий за п
7     - n_iters: int, default=10_000 — кол-во итераций метода М
8
9     Возвращает:
10    - results: list[bool] — массив результатов симуляций мето
11                                   True — игрок выиграл автомобиль,
12
13    '''
14    results = []
15    doors = [1, 2, 3]
16    for _ in range(n_iters):
17
18        door_auto = # напишите выбор случайной двери из 3 с помощью
19        door_player = # напишите выбор случайной двери из 3 с помо
20        # двери, которые может открыть ведущий:
21        doors_may_open = [door for door in doors if (door != door_
22        door_opened = # напишите выбор случайной двери из тех, ко
23        if change:
24            new_door_player = # напишите, какую теперь выбирает де
25            door_player = new_door_player
26
27        results.append(int(door_player == door_auto))
28
29    return results
30
31 results_change = monty_hall_game(change=True)
32 results_not_change = monty_hall_game(change=False)
33
34 print(f'результат, когда меняем дверь: {np.round(<выведете процен
35 print(f'результат, когда не меняем дверь: {np.round(<выведете про

```

```

Ввод [6]: 1 def monty_hall_game(change=False, n_iters=10_000):
2     '''
3     Функция, симулирующая игру Монти-Холла с помощью метода Монте-Карло
4
5     Параметры:
6     - change: bool, default=False — параметр, отвечающий за смену двери
7     - n_iters: int, default=10_000 — кол-во итераций метода Монте-Карло
8
9     Возвращает:
10    - results: list[bool] — массив результатов симуляций метода Монте-Карло
11    True — игрок выиграл автомобиль, False — проиграл
12
13    '''
14    results = []
15    doors = [1, 2, 3]
16    for _ in range(n_iters):
17        door_auto = np.random.choice(doors)
18        door_player = np.random.choice(doors)
19        # двери, которые может открыть ведущий:
20        doors_may_open = [door for door in doors if (door != door_auto)]
21        door_opened = np.random.choice(doors_may_open)
22        if change:
23            new_door_player = [door for door in doors if (door != door_opened)]
24            door_player = new_door_player
25
26        results.append(int(door_player == door_auto))
27
28    return results
29
30 results_change = monty_hall_game(change=True)
31 results_not_change = monty_hall_game(change=False)
32
33 print(f'результат, когда меняем дверь: {np.round(100 * np.mean(results_change), 1)}%')
34 print(f'результат, когда не меняем дверь: {np.round(100 * np.mean(results_not_change), 1)}%')

```

результат, когда меняем дверь: 65.8%
результат, когда не меняем дверь: 33.6%

Поразительно! В 2/3 случаев, когда мы меняем дверь, мы выигрываем, тогда как если не меняем, выигрываем только в 1/3 случаев.

Но как так? — Дело в том, что если не менять дверь, то вы выиграете с вероятностью, равной первоначальной, то есть, 1/3. При этом у вас есть 2 двери и 2 решения: менять дверь или нет, и где-то находится автомобиль (так как открыли дверь с козой). Значит, если вероятность выиграть без смены двери 1/3, то вероятность со сменой двери — 2/3!

Но почему 2/3? — Потому что исключение одной двери перенесло всю вероятность авто в ту, на которую вы можете поменять. Более формально — можно покопаться в условных вероятностях и получить требуемые 1/3 и 2/3 вероятности.

2.3. Ошибка выжившего

В одну американскую больницу направляли всех пострадавших в результате несчастных случаев в городе. Но больше всего в больницу попадало водителей и пассажиров, пострадавших в ДТП. Чтобы уменьшить их количество, городские власти сделали обязательным пользование ремнями безопасности. Водители и пассажиры стали пристегиваться ремнями. После этого число ДТП осталось неизменным, но вот число пострадавших в них людей, которые поступали в больницу, даже увеличилось. Вопрос: Почему?

На первый взгляд зависимость такая: начали пристегиваться → стало больше пострадавших → ремни неэффективны

Вопрос в зал: что произошло на самом деле?

► Сведения

2.4. Парадокс полицейского

2.4.1. Пререквизит: условная вероятность

- **Вероятность события A , если известно, что произошло событие B :**

$$P(A|B) = \frac{P(A \cap B)}{P(B)}, P(B) > 0.$$

Пример: вероятность, что выпадет 6, если известно, что выпало четное число.

- **Независимость событий:** события A и B **независимы**, если $P(A \cap B) = P(A) \cdot P(B)$.

Пример: вероятность, что выпадет 6, если известно, что в прошлый раз выпало 3.

Замечание: независимость еще можно определить как $P(A|B) = P(A)$, если $P(B) \neq 0$.

- **Задача на условные вероятности:**

Задача: Игральные кости

Бросают две игральные кости. Известно, что сумма выпавших очков равна 8.

Вопрос:

- Какова вероятность того, что на первой кости выпало число 5?

Решение:

1. Пространство элементарных событий:

Всего возможных пар исходов при броске двух костей — $6 \times 6 = 36$.

2. Условие:

Рассматриваем только те исходы, где сумма очков равна 8:

(2, 6), (3, 5), (4, 4), (5, 3), (6, 2).

Таким образом, всего таких исходов 5.

3. Интересующее событие:

На первой кости выпало 5. Подходящий исход — (5, 3).

4. Условная вероятность:

По формуле условной вероятности:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}.$$

Вероятность события B (сумма равна 8):

$$P(B) = \frac{\text{Число подходящих исходов}}{\text{Общее число исходов}} = \frac{5}{36}.$$

Вероятность события $A \cap B$ (первая кость — 5, сумма — 8):

$$P(A \cap B) = \frac{1}{36}.$$

Условная вероятность:

$$P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{\frac{1}{36}}{\frac{5}{36}} = \frac{1}{5}.$$

Ответ:

Вероятность того, что на первой кости выпало число 5, если известно, что сумма равна 8, равна $\frac{1}{5}$.

2.4.2. Пререквизит: формула Байеса и формула полной вероятности

Теорема: (формула Байеса) $P(A|B) = \frac{P(B|A)P(A)}{P(B)}.$

Доказательство: по определению условной вероятности:

$$P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{P(B|A)P(A)}{P(B)}.$$

Теорема: (формула полной вероятности) если $A = \sqcup A_i$, то есть, A состоит из объединения непересекающихся событий A_i , то $P(B) = \sum P(B|A_i)P(A_i).$

Доказательство: так как A состоит из объединения непересекающихся событий A_i , то $B = \sum P(B \cap A_i)$, а каждое $P(B \cap A_i) = P(B|A_i)P(A_i)$ по определению условной вероятности.

Следствие: $P(A_j|B) = \frac{P(B|A_j)P(A_j)}{\sum P(B|A_i)P(A_i)}.$

2.4.3. Непосредственно, парадокс полицейского

- Алкотестер почти идеален:
 - Точность при наличии алкоголя: 100% (если человек пьян, алкотестер это определит всегда)
 - Точность при отсутствии алкоголя: 98% (если человек не пьян, алкотестер в 2% случая скажет, что он пьян)
- Реально пьян только каждый тысячный человек
Полицейский проверил случайного человека и алкотестер показал, что он пьян
Полицейский сказал, что этот человек пьян с вероятностью 98% и взял штраф с человека

Вопрос в зал: прав ли полицейский?

► Сведения

2.5. Парадокс друзей

Почему твои друзья популярнее, чем ты (если ты не Кайли Дженнер)

Согласно данным ученых из университета Южной Калифорнии, более, чем 98% пользователей твиттера имеют меньше друзей, чем их друзья в среднем.

Вопрос в зал: с чем же это связано?

► Сведения

Ввод [7]:

```
1 from graphviz import Graph as GVGraph
2 from itertools import permutations
```

```
Ввод [8]: 1 np.random.seed(seed=42)
2
3 persons = ['Алиса', 'Борис', 'Виталий', 'Геннадий', 'Елена', 'Жанна']
4 # генерируем некоторую меру дружбы для каждого из компании
5 p_friendship = sps.beta(a=0.8, b=0.8).rvs(size=len(persons))
6 probs = dict(zip(persons, p_friendship))
7
8 # генерируем граф дружбы
9 relations = []
10 for pair in permutations(persons, 2):
11     if pair[0] < pair[1]:
12         # вероятность дружбы – среднее арифметическое мер дружбы 2
13         friendship = sps.bernoulli((probs[pair[0]] + probs[pair[1]])/2)
14         if friendship == 1:
15             relations.append(pair)
16
17 relations
```

```
Out [8]: [('Алиса', 'Геннадий'),
          ('Алиса', 'Елена'),
          ('Алиса', 'Зиновий'),
          ('Алиса', 'Илья'),
          ('Борис', 'Геннадий'),
          ('Борис', 'Зиновий'),
          ('Борис', 'Илья'),
          ('Виталий', 'Геннадий'),
          ('Геннадий', 'Елена'),
          ('Геннадий', 'Жанна'),
          ('Геннадий', 'Зиновий'),
          ('Елена', 'Илья'),
          ('Жанна', 'Илья'),
          ('Зиновий', 'Илья')]
```


Ввод [9]:

```

1  class Vertex:
2      """
3      Класс представляет вершину графа.
4
5      Атрибуты:
6          name (str): имя вершины (уникальный идентификатор).
7          neighbors (set[Vertex]): множество соседних вершин.
8
9      Методы:
10         add_neighbor(vertex): добавляет вершину в список соседей.
11         degree(): возвращает количество соседей (степень вершины)
12         get_neighbors(): возвращает список соседних вершин.
13         get_neighbor_by_name(name): ищет соседа по имени.
14     """
15
16     def __init__(self, name):
17         """
18         Создает вершину с заданным именем и пустым множеством сос
19
20         Args:
21             name (str): имя вершины.
22         """
23         self.name = name
24         self.neighbors = set()
25
26     def add_neighbor(self, vertex):
27         """
28         Добавляет вершину vertex в список соседей текущей вершины
29
30         Args:
31             vertex (Vertex): соседняя вершина.
32         """
33         self.neighbors.add(vertex)
34
35     def degree(self):
36         """
37         Возвращает количество соседей (степень вершины).
38
39         Returns:
40             int: число соседей.
41         """
42         return len(self.neighbors)
43
44     def get_neighbors(self):
45         """
46         Возвращает список соседних вершин.
47
48         Returns:
49             list[Vertex]: список соседей.
50         """
51         return list(self.neighbors)
52
53     def get_neighbor_by_name(self, name):
54         """
55         Ищет среди соседей вершину с указанным именем.
56
57         Args:
58             name (str): имя искомой вершины.
59
60         Returns:
61             Vertex | None: вершина с заданным именем или None, ес

```

```

62         """
63         for neighbor in self.neighbors:
64             if neighbor.name == name:
65                 return neighbor
66         return None
67
68     def __repr__(self):
69         """Возвращает строковое представление вершины."""
70         return f"Vertex({self.name})"
71
72
73 class Graph:
74     """
75     Класс представляет неориентированный граф.
76
77     Атрибуты:
78         vertices (dict[str, Vertex]): словарь, где ключ — имя вер
79
80     Методы:
81         from_edge_list(edge_list): создает граф на основе списка
82         add_vertex(name): добавляет вершину по имени.
83         add_edge(from_name, to_name): добавляет ребро между верши
84         get_vertex(name): возвращает вершину по имени.
85         visualize(): строит визуализацию графа через Graphviz.
86     """
87
88     def __init__(self):
89         """
90         Создает пустой граф.
91         """
92         self.vertices = {}
93
94     @classmethod
95     def from_edge_list(cls, edge_list):
96         """
97         Создает граф на основе списка рёбер.
98
99         Args:
100             edge_list (list[tuple[str, str]]): список рёбер в фор
101
102         Returns:
103             Graph: объект графа.
104         """
105         graph = cls()
106         for from_v, to_v in edge_list:
107             graph.add_edge(from_v, to_v)
108         return graph
109
110     def add_vertex(self, name):
111         """
112         Добавляет вершину с именем name в граф, если она ещё не с
113
114         Args:
115             name (str): имя вершины.
116
117         Returns:
118             Vertex: объект вершины.
119         """
120         if name not in self.vertices:
121             self.vertices[name] = Vertex(name)
122         return self.vertices[name]

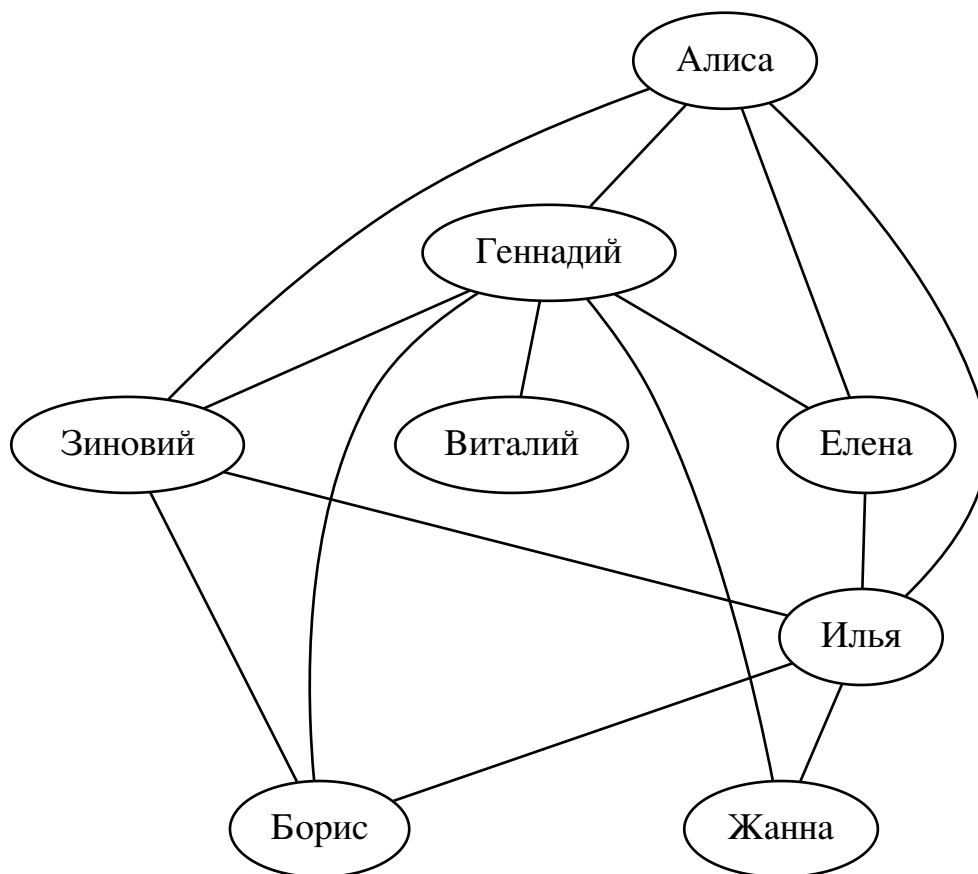
```

```
123
124 def add_edge(self, from_name, to_name):
125     """
126     Добавляет ребро между вершинами from_name и to_name.
127     Если вершины ещё не существуют, они будут автоматически д
128
129     Args:
130         from_name (str): имя первой вершины.
131         to_name (str): имя второй вершины.
132     """
133     from_vertex = self.add_vertex(from_name)
134     to_vertex = self.add_vertex(to_name)
135     from_vertex.add_neighbor(to_vertex)
136     to_vertex.add_neighbor(from_vertex)
137
138 def get_vertex(self, name):
139     """
140     Возвращает вершину по имени.
141
142     Args:
143         name (str): имя вершины.
144
145     Returns:
146         Vertex | None: вершина с заданным именем или None, ес
147     """
148     return self.vertices.get(name, None)
149
150 def visualize(self):
151     """
152     Создает объект GVGraph для визуализации графа (например,
153     Добавляет все вершины и рёбра, исключая дублирование.
154
155     Returns:
156         GVGraph: объект для построения визуализации.
157     """
158     dot = GVGraph()
159     added_edges = set()
160     for vertex in self.vertices.values():
161         dot.node(vertex.name)
162         for neighbor in vertex.neighbors:
163             edge = tuple(sorted((vertex.name, neighbor.name)))
164             if edge not in added_edges:
165                 dot.edge(vertex.name, neighbor.name)
166                 added_edges.add(edge)
167     return dot
168
169 def __repr__(self):
170     """Возвращает строковое представление графа."""
171     return f"Graph(vertices={list(self.vertices.keys())})"
```



```
Ввод [10]: 1 friends_graph = Graph.from_edge_list(relations)
           2 friends_graph.visualize()
```

Out[10]:



```
Ввод [11]: 1 friends_graph.vertices
```

Out[11]:

```
{'Алиса': Vertex(Алиса),
 'Геннадий': Vertex(Геннадий),
 'Елена': Vertex(Елена),
 'Зиновий': Vertex(Зиновий),
 'Илья': Vertex(Илья),
 'Борис': Vertex(Борис),
 'Виталий': Vertex(Виталий),
 'Жанна': Vertex(Жанна)}
```

Практика: дополните код для подсчета друзей

Ввод []:

```
1 # создаем пустой датафрейм
2 cnt_friends_df = pd.DataFrame(columns=['имя', 'кол-во друзей', 'ср
3
4 # проходимся по людям
5 for person in persons:
6     vertex_person = # вызовете метод, который по имени достает вер
7     cnt_person = # вызовете метод, который по вершине достает кол-
8     neighbours = # вызовете метод, который по вершине достает сосе
9     cnt_friends_of_friend = []
10    # проходимся по соседям
11    for neighbour in neighbours:
12        cnt_friends_of_friend.append(# вызовете метод, который по
13
14    avg_cnt_friends_of_friend = # вызовете функцию из пипру для по
15    cnt_friends_row = pd.DataFrame(data={'имя': [person],
16                                         'кол-во друзей': [cnt_pe
17                                         'среднее кол-во друзей у
18    cnt_friends_df = pd.concat([cnt_friends_df, cnt_friends_row],
19
20 cnt_friends_df
```

Ввод [13]:

```

1 # создаем пустой датафрейм
2 cnt_friends_df = pd.DataFrame(columns=['имя', 'кол-во друзей', 'ср
3
4 # проходимся по людям
5 for person in persons:
6     vertex_person = friends_graph.get_vertex(person)
7     cnt_person = vertex_person.degree()
8     neighbours = vertex_person.get_neighbors()
9     cnt_friends_of_friend = []
10    # проходимся по соседям
11    for neighbour in neighbours:
12        cnt_friends_of_friend.append(neighbour.degree())
13
14    avg_cnt_friends_of_friend = np.mean(cnt_friends_of_friend)
15    cnt_friends_row = pd.DataFrame(data={'имя': [person],
16                                         'кол-во друзей': [cnt_per
17                                         'среднее кол-во друзей у
18    cnt_friends_df = pd.concat([cnt_friends_df, cnt_friends_row],
19
20 cnt_friends_df

```

Out [13]:

	имя	кол-во друзей	среднее кол-во друзей у друзей
0	Алиса	4	4.500000
1	Борис	3	5.000000
2	Виталий	1	6.000000
3	Геннадий	6	2.833333
4	Елена	3	5.000000
5	Жанна	2	5.500000
6	Зиновий	4	4.500000
7	Илья	5	3.200000

Ввод [14]:

```

1 cnt = len(cnt_friends_df[cnt_friends_df['кол-во друзей'] > cnt_fr
2
3 print(f'y {100 * np.round(1 - cnt / len(persons), 2)}% друзей мене

```

у 75.0% друзей меньше, чем у среднего их друга

Разберем пример попроще: пусть в школе есть 2 класса: в одном 100 человек, а во втором — 10 человек. У каждого ученика в школе спросили, сколько человек учится в его классе, а потом усреднили. Получилось $(100 \cdot 100 + 10 \cdot 10) / (100 + 10) \approx 92$ человека, хотя на самом деле среднее число учеников в классе $(100 + 10) / 2 = 55$. Смещение образовалось из-за того, что чаще спрашивали тех, кто учится в большем классе (так как их просто больше).

Вывод: Парадокс дружбы — это пример взвешенного среднего, где вес — это число связей у человека. Люди с большим числом связей оказывают непропорционально большое влияние на статистику, поэтому смещают ее в большую сторону.

Где еще можно встретиться с парадоксом дружбы:

